Aluno: Vinicius Ramon Ferreira **R.A:** 24360978-2

Atividade - Aula 009

Exercício em duplas (10 min): qual método é chamado? (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

1 · Java

```
class Animal {
  String som() { return "..."; }
}
class Cachorro extends Animal {
  String som() { return "au"; }
}
Animal x = new Cachorro();
System.out.println(x.som());
```

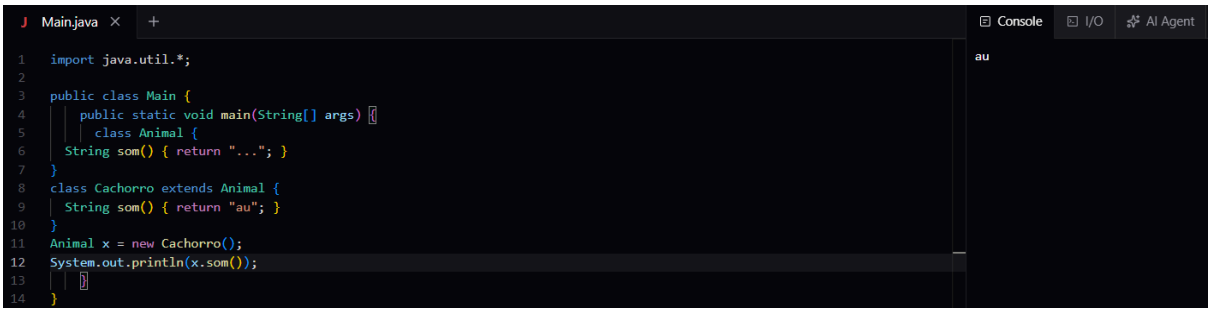
2 · C++

```
struct A {
  void f() { cout << "A"; }
};
struct B : A {
  void f() { cout << "B"; }
};
B b; A *p = &b;
p->f();
```

Arquivos: exemplos/08_exercicio · execute em onecompiler.com

1 - Java

(a) Saída:



au

(b) Decisão: execução, porque: som() é um método de instância sobrescrito em Cachorro. O tipo declarado de x é Animal, mas o objeto criado é um Cachorro. O compilador só confere que Animal possui som(). Em tempo de execução, a JVM escolhe a versão pelo tipo real do objeto (despacho dinâmico), então roda Cachorro.som().

2 - C++

(a) Saída:



A

(b) Decisão: compilação, porque: f() não é virtual, então não há despacho dinâmico. O ponteiro p é do tipo A*, e o compilador liga a chamada a A::f (ligação estática), mesmo que p aponte para um B. Com virtual, a saída seria B e a decisão passaria para a execução.

Exercício: qual método é chamado? (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

3 · Java

```
class A { String nome = "A";
  String getNome() { return nome; } }
class B extends A { String nome = "B";
  String getNome() { return nome; } }
A x = new B();
System.out.println(x.nome + " " + x.getNome());
```

4 · Python

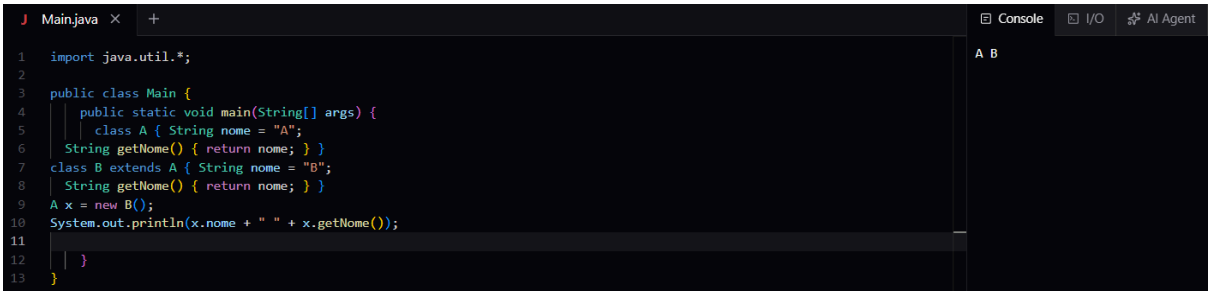
```
class Contador:
    total = 0
    def __init__(self):
        Contador.total += 1
        self.id = Contador.total

a = Contador(); b = Contador()
print(a.id, b.id, a.total)
```

Arquivos: exemplos/08_exercicio · execute em onecompiler.com

3 - Java

(a) Saída:



A B

(b) Decisão: x.nome é decidido na compilação e x.getNome() na execução, porque: atributos não são polimórficos. x.nome usa o tipo declarado (A), então vale "A". getNome() é método sobrescrito e usa o objeto real (B), então retorna "B". A classe B esconde o campo (field hiding), mas não o sobrescreve.

4 - Python
(a) Saída:



1 2 2
(b) Decisão: execução, porque: total é um atributo de classe, compartilhado por todas as instâncias. Cada __init__ incrementa Contador.total e guarda o valor em self.id. Assim, a.id = 1 e b.id = 2. Como a não tem total próprio, o Python busca na classe e encontra 2. Python não tem tipagem estática, então toda busca de atributo e método acontece na execução.

Exercício: qual método é chamado? (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

5 · Java

```
class A {
  static String quem() { return "A"; }
}
class B extends A {
  static String quem() { return "B"; }
}
A x = new B();
System.out.println(x.quem());
```

6 · Go

```
type Animal struct{}
func (Animal) Som() string { return "..." }
func (a Animal) Falar() string {
    return "faz " + a.Som()
}
type Cao struct{ Animal }
func (Cao) Som() string { return "au" }

fmt.Println(Cao{}.Falar(), Cao{}.Som())
```

Arquivos: exemplos/08_exercicio · execute em onecompiler.com

5 - Java
(a) Saída:



A
(b) Decisão: compilação, porque: métodos static pertencem à classe, não ao objeto. Eles não são sobrescritos, e sim escondidos (method hiding). Não há despacho dinâmico, então o compilador usa o tipo declarado de x (A) e liga a chamada a A.quem().

6 - Go

(a) Saída:



faz ... au

(b) Decisão: compilação, porque: Go não tem herança. Cao apenas embute Animal, e os métodos de Animal são promovidos. Cao{}.Som() chama Cao.Som e retorna "au". Já Cao{}.Falar() chama o Falar promovido de Animal, em que o receptor a é um Animal de verdade, então a.Som() chama Animal.Som e retorna "...", resultando em "faz ...". Não existe "this" polimórfico, e os tipos são concretos, então tudo é resolvido na compilação. O despacho dinâmico em Go só ocorreria via interfaces.